# DecSync Community Platform

## System Architecture & Design Document

**Version 1.0**

---

# Table of Contents



---

# 1. Introduction

## 1.1 Purpose

This document describes the complete system architecture, design decisions, and implementation plan for the DecSync Community Platform — a collaborative learning space built specifically for members of the DecSync batch. The platform enables resource sharing, technical blogging, community discussions, and DSA practice in a single unified interface.

## 1.2 Design Principles

- **Zero cost during development.** Every tool, service, and platform used must have a free tier that comfortably supports a community of 50–300 students without requiring a credit card or trial account.
- **Simplicity first.** The architecture starts as a well-organized monolith. Complexity is introduced only when genuinely needed.
- **No self-hosted file storage.** All files (PDFs, notes, videos) are stored on user-managed Google Drive or YouTube. The platform stores only metadata and URLs.
- **Incrementally extensible.** Features are built in phases. The schema and API are designed so that later phases plug in cleanly without breaking existing work.

## 1.3 Scope

The platform covers the following functional areas:

- User registration, login, and profile management
- Resource upload and discovery
- Blog writing and reading
- Community forum discussions with nested comments
- Reputation and achievement system
- Moderation and admin tools
- Announcements board
- DSA problem lists with external links
- Global search across all content

---

# 2. Technology Stack

Every item in this stack is free to use for a project of this scale.

| Layer | Technology | Reason |
|---|---|---|
| Frontend | Next.js (App Router) | SSR, file-based routing, API routes built in |
| Styling | Tailwind CSS | Utility-first, no design system cost |
| Database | Supabase (PostgreSQL) | 500MB free, built-in auth, realtime, RLS |

| Layer | Technology | Reason |
|---|---|---|
| Authentication | Supabase Auth | Email/password, email verification, JWT |
| API Layer | Next.js API Routes | Serverless, co-located with frontend |
| Rich Text Editor | TipTap | Open source, Markdown + code block support |
| Syntax Highlighting | Shiki | Free, server-side, accurate highlighting |
| Search | PostgreSQL Full-Text Search | Built into Supabase, no extra infra needed |
| Real-time | Supabase Realtime | WebSocket-based, free tier: 200 connections |
| Hosting | Vercel (Hobby Plan) | 100GB bandwidth/month, no credit card needed |
| Email (auth only) | Supabase Built-in SMTP | 3 emails/hour, sufficient for small community |
| External File Storage | Google Drive / YouTube | User-managed, platform stores links only |

---

# 3. System Architecture

## 3.1 Architecture Style

The platform uses a **monolithic Next.js application** with a clear internal separation of concerns. The frontend (React pages and components) and the backend (API routes) live in the same codebase and are deployed together on Vercel. The database and authentication are fully managed by Supabase.

This avoids the operational overhead of running and maintaining separate services while still keeping the codebase organized enough to extract services later if the platform grows significantly.

## 3.2 High-Level Architecture Diagram

1. The User's Browser handles the Next.js pages, React components, and session tokens via cookies.
2. HTTPS Requests are sent to Vercel (Free Tier) which hosts the Next.js Frontend and API Routes.
3. The API layer communicates securely with Supabase (Free Tier) via the JS Client.
4. Supabase manages Authentication, the PostgreSQL database, and Realtime updates.
5. External storage (Google Drive, YouTube, GitHub) is referenced via URLs, so files never touch the platform servers.

## 3.3 Request Lifecycle

A typical user request flows as follows:

1. User opens a page (e.g. /resources)
2. Next.js server component runs on Vercel serverless function
3. Server component calls Supabase directly using the service key
4. Supabase executes the query against PostgreSQL, applies RLS policies
5. Data is returned to the server component, which renders HTML
6. HTML is sent to the browser — page appears immediately (SSR)
7. React hydrates the page, making it interactive
8. Subsequent interactions (votes, comments) hit /api/* routes client-side
9. API routes validate the JWT from the user's cookie
10. Validated requests are executed against Supabase and results returned

---

# 4. Authentication System

## 4.1 Overview

Authentication is handled entirely by Supabase Auth using email and password. There is no Google Sign-In or any other OAuth provider. Users register with their name, email, password, and batch. Email verification is required before accessing the platform.

## 4.2 Auth Pages

| | |
|---|---|
| /auth/signup | Registration form (name, email, password, batch) |
| /auth/login | Login form (email, password) |
| /auth/verify | Confirmation screen shown after signup |

| | |
|---|---|
| /auth/forgot | Form to request a password reset email |
| /auth/reset | New password form (reached via emailed link) |

## 4.3 Sign Up Flow

1. User fills out the Signup Form with Name, Email, Password, and Batch.
2. The Frontend validates the input using a Zod schema (required fields, 8-char password, email format).
3. The client performs a POST request to /api/auth/signup.
4. The API Route performs server-side Zod validation.
5. Supabase auth.signUp is called with user metadata.
6. Supabase creates the auth record, sends a verification email, and triggers a public profile insertion.
7. The browser displays a verification message.
8. User clicks the link in their email; Supabase verifies the account and redirects to the dashboard.

## 4.4 Login Flow

1. User enters email and password into the Login Form.
2. Supabase signInWithPassword is triggered.
3. If successful, JWT access and refresh tokens are stored in an httpOnly cookie.
4. If failed, the form displays an "Invalid email or password" error.
5. On success, the user is redirected to the home page or their previous destination.

## 4.5 Password Reset Flow

1. User clicks "Forgot Password" and enters their email.
2. Supabase resetPasswordForEmail is called, sending a link to the user.
3. User clicks the link and lands on the /auth/reset page.
4. User enters a new password.
5. Supabase updateUser saves the new password.
6. User is redirected to the login page with a success message.

## 4.6 Session Management

The `@supabase/ssr` package is used to correctly handle Supabase sessions in Next.js. Tokens are stored in cookies and are available in both server components and client

components. The access token expires every hour and is silently refreshed using the refresh token (valid for 7 days).

Server Component:
  createServerClient(url, key, { cookies })
  supabase.auth.getUser()

Client Component:
  createBrowserClient(url, key)
  supabase.auth.getSession()

## 4.7 Route Protection Middleware

middleware.ts (runs before every request on Vercel)

Protected routes:
  /profile/edit
  /resources/upload
  /blogs/write
  /forums/new
  /admin/*

Logic:
  Read session from cookies
  If no session and route is protected → redirect to /auth/login?next=[path]
  If session exists but role is not admin and route is /admin/* → redirect to /403
  Otherwise → allow request through

## 4.8 Database Trigger for New Users

When Supabase creates a new auth user, this trigger automatically creates the corresponding public profile:

```
CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.users (id, email, name, batch)
  VALUES (
    NEW.id,
    NEW.email,
    NEW.raw_user_meta_data->>'name',
    NEW.raw_user_meta_data->>'batch'
```

```
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE PROCEDURE handle_new_user();
```

---

# 5. Database Schema

All tables live in Supabase's PostgreSQL instance. Row-Level Security (RLS) is enabled on every table. Foreign key relationships are enforced at the database level.

## 5.1 Entity Relationship Overview

```
users
  |
  +--< resources          (one user uploads many resources)
  |
  +--< blog_posts         (one user writes many blogs)
  |
  +--< forum_threads      (one user creates many threads)
  |
  +--< forum_comments     (one user writes many comments)
  |       |
  |       +--< forum_comments   (self-referencing for nesting)
  |
  +--< votes              (one user casts many votes)
  |
  +--< notifications      (one user receives many notifications)
  |
  +--< achievements       (one user earns many achievements)
  |
  +--< reports            (one user files many reports)

forum_threads
  |
  +--< forum_comments     (one thread has many comments)

announcements             (standalone, only admins/mods create)
```

dsa_problems (standalone, curated by admins)
dsa_progress (user <-> problem, tracks completion)

## 5.2 Table Definitions

### users

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY (from auth.users) |
| email | VARCHAR | UNIQUE, NOT NULL |
| name | VARCHAR | NOT NULL |
| batch | VARCHAR | e.g. "DecSync 2024" |
| bio | TEXT | |
| avatar_url | TEXT | external URL only |
| role | VARCHAR | DEFAULT 'student', CHECK IN ('student','moderator','admin','banned') |
| reputation | INTEGER | DEFAULT 0 |
| created_at | TIMESTAMPTZ | DEFAULT now() |

### resources

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| title | VARCHAR | NOT NULL |
| description | TEXT | |
| author_id | UUID | REFERENCES users(id) ON DELETE SET NULL |
| category | VARCHAR | CHECK IN ('notes','pdf','placement','dsa', ...) |

| Column | Type | Constraints |
|---|---|---|
| semester | INTEGER | |
| subject | VARCHAR | |
| tags | TEXT[] | DEFAULT '{}' |
| external_url | TEXT | NOT NULL |
| thumbnail_url | TEXT | |
| upvotes | INTEGER | DEFAULT 0 |
| search_vector | TSVECTOR | auto-updated by trigger |
| created_at | TIMESTAMPTZ | DEFAULT now() |

INDEX: GIN index on search_vector
INDEX: GIN index on tags

## blog_posts

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| title | VARCHAR | NOT NULL |
| slug | VARCHAR | UNIQUE NOT NULL |
| author_id | UUID | REFERENCES users(id) ON DELETE SET NULL |
| content | TEXT | stored as HTML (sanitized) |
| excerpt | TEXT | |
| tags | TEXT[] | DEFAULT '{}' |
| status | VARCHAR | DEFAULT 'draft', CHECK IN ('draft','published') |
| views | INTEGER | DEFAULT 0 |
| upvotes | INTEGER | DEFAULT 0 |

| Column | Type | Constraints |
|---|---|---|
| search_vector | TSVECTOR | auto-updated by trigger |
| created_at | TIMESTAMPTZ | DEFAULT now() |
| updated_at | TIMESTAMPTZ | DEFAULT now() |

INDEX: GIN index on search_vector
UNIQUE INDEX: on slug

## forum_threads

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| title | VARCHAR | NOT NULL |
| author_id | UUID | REFERENCES users(id) ON DELETE SET NULL |
| category | VARCHAR | CHECK IN ('dsa','development','placements', ...) |
| body | TEXT | |
| upvotes | INTEGER | DEFAULT 0 |
| is_locked | BOOLEAN | DEFAULT FALSE |
| is_pinned | BOOLEAN | DEFAULT FALSE |
| views | INTEGER | DEFAULT 0 |
| search_vector | TSVECTOR | auto-updated by trigger |
| created_at | TIMESTAMPTZ | DEFAULT now() |

INDEX: GIN index on search_vector

## forum_comments

Column Type Constraints
--- --- ---
id UUID PRIMARY KEY DEFAULT gen_random_uuid()

thread_id  UUID  REFERENCES forum_threads(id) ON DELETE CASCADE
parent_id  UUID  REFERENCES forum_comments(id) ON DELETE CASCADE
                 NULLABLE (null = top-level comment)
author_id  UUID  REFERENCES users(id) ON DELETE SET NULL
content  TEXT  NOT NULL
upvotes  INTEGER  DEFAULT 0
is_deleted  BOOLEAN  DEFAULT FALSE
created_at  TIMESTAMPTZ  DEFAULT now()

INDEX: on thread_id
INDEX: on parent_id

### votes

Column  Type  Constraints
---  ---  ---
id  UUID  PRIMARY KEY DEFAULT gen_random_uuid()
user_id  UUID  REFERENCES users(id) ON DELETE CASCADE
target_id  UUID  NOT NULL
target_type  VARCHAR  CHECK IN ('resource','blog','thread','comment')
value  INTEGER  CHECK IN (1, -1)
created_at  TIMESTAMPTZ  DEFAULT now()

UNIQUE: (user_id, target_id, target_type)  -- one vote per user per item

### notifications

Column  Type  Constraints
---  ---  ---
id  UUID  PRIMARY KEY DEFAULT gen_random_uuid()
user_id  UUID  REFERENCES users(id) ON DELETE CASCADE
type  VARCHAR  CHECK IN ('reply','mention',
                 'new_resource','announcement')
message  TEXT  NOT NULL
link  TEXT
is_read  BOOLEAN  DEFAULT FALSE
created_at  TIMESTAMPTZ  DEFAULT now()

INDEX: on (user_id, is_read)

### achievements

Column  Type  Constraints

| | | |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| user_id | UUID | REFERENCES users(id) ON DELETE CASCADE |
| badge_key | VARCHAR | e.g. 'first_blog', '10_answers', 'streak_30' |
| earned_at | TIMESTAMPTZ | DEFAULT now() |

UNIQUE: (user_id, badge_key)  -- earn each badge only once

### reports

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| reporter_id | UUID | REFERENCES users(id) ON DELETE SET NULL |
| target_id | UUID | NOT NULL |
| target_type | VARCHAR | CHECK IN ('blog','thread','comment','resource') |
| reason | TEXT | NOT NULL |
| status | VARCHAR | DEFAULT 'pending' CHECK IN ('pending','resolved','dismissed') |
| created_at | TIMESTAMPTZ | DEFAULT now() |

### announcements

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| author_id | UUID | REFERENCES users(id) ON DELETE SET NULL |
| title | VARCHAR | NOT NULL |
| body | TEXT | NOT NULL |
| type | VARCHAR | CHECK IN ('placement','internship', 'hackathon','event','deadline','general') |
| expires_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | DEFAULT now() |

### dsa_problems

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY DEFAULT gen_random_uuid() |
| title | VARCHAR | NOT NULL |
| topic | VARCHAR | e.g. 'Arrays', 'Graphs', 'DP' |
| difficulty | VARCHAR | CHECK IN ('easy','medium','hard') |
| platform | VARCHAR | CHECK IN ('leetcode','codeforces', |

'codechef','gfg','other')
external_url TEXT NOT NULL
week_number INTEGER for weekly challenge grouping
is_weekly BOOLEAN DEFAULT FALSE
created_at TIMESTAMPTZ DEFAULT now()

### dsa_progress

Column Type Constraints
---
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id UUID REFERENCES users(id) ON DELETE CASCADE
problem_id UUID REFERENCES dsa_problems(id) ON DELETE CASCADE
status VARCHAR CHECK IN ('attempted','solved')
solved_at TIMESTAMPTZ

UNIQUE: (user_id, problem_id)

---

# 6. API Design

All API routes live under `/api/` in the Next.js project. Every route validates the incoming request body using Zod. Routes that modify data require a valid session JWT, extracted from the cookie by the Supabase SSR client.

## 6.1 Response Format

All responses follow this consistent shape:

Success:
```
{
  "success": true,
  "data": { ... }
}
```

Error:
```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "You must be logged in to do this."
```

```
  }
}
```

## 6.2 Auth Routes

```
POST   /api/auth/signup
       Body: { name, email, password, batch }
       Response: { message: "Verification email sent" }

POST   /api/auth/login
       Body: { email, password }
       Response: { user: {...} }  + sets session cookie

POST   /api/auth/logout
       Response: { success: true }  + clears session cookie

POST   /api/auth/forgot
       Body: { email }
       Response: { message: "Reset email sent if account exists" }

POST   /api/auth/reset
       Body: { password }  (session from reset link required)
       Response: { message: "Password updated" }

GET    /api/auth/me
       Headers: session cookie
       Response: { user: { id, name, email, role, reputation, ... } }
```

## 6.3 Resource Routes

```
GET    /api/resources
       Query params:
         category    filter by category
         semester    filter by semester number
         subject     filter by subject name
         tags        comma-separated tag list
         sort        "newest" | "popular"
         page        pagination (default 1)
         limit       items per page (default 20)
       Response: { data: [...resources], total, page, limit }

POST   /api/resources
```

Auth: required
Body: { title, description, category, semester,
subject, tags, external_url, thumbnail_url }
Response: { data: { ...newResource } }

GET /api/resources/:id
Response: { data: { ...resource, author: {...} } }

PUT /api/resources/:id
Auth: required (owner or admin)
Body: { title, description, tags, ... }
Response: { data: { ...updatedResource } }

DELETE /api/resources/:id
Auth: required (owner or admin)
Response: { success: true }

POST /api/resources/:id/vote
Auth: required
Body: { value: 1 | -1 }
Response: { newCount: number }

## 6.4 Blog Routes

GET /api/blogs
Query params:
tags filter by tags
author filter by author user ID
status "published" only for public, "draft" for own drafts
sort "newest" | "popular"
page, limit
Response: { data: [...posts], total, page, limit }

POST /api/blogs
Auth: required
Body: { title, content, excerpt, tags, status }
Response: { data: { ...newPost, slug } }

GET /api/blogs/:slug
Response: { data: { ...post, author: {...} } }
Side effect: increments views counter

PUT /api/blogs/:id

Auth: required (owner only)
Body: { title, content, excerpt, tags, status }
Response: { data: { ...updatedPost } }

DELETE /api/blogs/:id
Auth: required (owner or admin)
Response: { success: true }

## 6.5 Forum Routes

GET /api/forums
Query params:
category filter by category
sort "hot" | "new" | "top"
page, limit
Response: { data: [...threads], total, page, limit }

POST /api/forums
Auth: required
Body: { title, body, category }
Response: { data: { ...newThread } }

GET /api/forums/:id
Response: { data: { ...thread, comments: [...nestedTree] } }
Side effect: increments views counter

PUT /api/forums/:id
Auth: required (owner only, if not locked)
Body: { title, body }
Response: { data: { ...updatedThread } }

DELETE /api/forums/:id
Auth: required (admin or moderator)
Response: { success: true }

PATCH /api/forums/:id/lock
Auth: required (admin or moderator)
Body: { locked: true | false }
Response: { success: true }

PATCH /api/forums/:id/pin
Auth: required (admin or moderator)
Body: { pinned: true | false }

Response: { success: true }

POST /api/forums/:id/vote
Auth: required
Body: { value: 1 | -1 }
Response: { newCount: number }

POST /api/forums/:id/comments
Auth: required
Body: { content, parent_id (optional) }
Response: { data: { ...newComment } }

POST /api/comments/:id/vote
Auth: required
Body: { value: 1 | -1 }
Response: { newCount: number }

DELETE /api/comments/:id
Auth: required (owner or admin)
Response: { success: true }
Note: Sets is_deleted = true, content replaced with "[deleted]"

## 6.6 User Routes

GET /api/users/:id
Response: { data: { ...publicProfile,
stats: { blogs, resources, threads, reputation },
achievements: [...] } }

PUT /api/users/:id
Auth: required (self only)
Body: { name, bio, avatar_url, batch }
Response: { data: { ...updatedUser } }

GET /api/users/:id/blogs
Response: { data: [...publishedBlogs] }

GET /api/users/:id/resources
Response: { data: [...resources] }

GET /api/users/:id/threads
Response: { data: [...threads] }

## 6.7 Search Route

GET /api/search
Query params:
q search query string (required)
type "all" | "resource" | "blog" | "forum" | "user"
default: "all"
page, limit
Response:

```
{
  data: {
    resources: [...],
    blogs: [...],
    forums: [...],
    users: [...]
  },
  total: number
}
```

## 6.8 Notification Routes

GET /api/notifications
Auth: required
Response: { data: [...notifications], unreadCount: number }

PATCH /api/notifications/:id/read
Auth: required
Response: { success: true }

PATCH /api/notifications/read-all
Auth: required
Response: { success: true }

## 6.9 Admin Routes

GET /api/admin/reports
Auth: required (admin or moderator)
Query: status = "pending" | "resolved" | "dismissed"
Response: { data: [...reports] }

PATCH /api/admin/reports/:id
Auth: required (admin or moderator)

Body: { status: "resolved" | "dismissed" }
Response: { success: true }

DELETE /api/admin/users/:id
Auth: required (admin only)
Note: Sets role to "banned", does not delete user
Response: { success: true }

POST /api/admin/announcements
Auth: required (admin or moderator)
Body: { title, body, type, expires_at }
Response: { data: { ...newAnnouncement } }

---

# 7. Frontend Architecture

## 7.1 Project Structure

```
decsync/
|
+-- app/                      Next.js App Router pages
|   |
|   +-- page.tsx              Home / Dashboard
|   +-- layout.tsx            Root layout (navbar, footer)
|   |
|   +-- auth/
|   |   +-- signup/page.tsx
|   |   +-- login/page.tsx
|   |   +-- verify/page.tsx
|   |   +-- forgot/page.tsx
|   |   +-- reset/page.tsx
|   |
|   +-- resources/
|   |   +-- page.tsx          Resource listing
|   |   +-- upload/page.tsx     Upload form
|   |   +-- [id]/page.tsx       Resource detail
|   |
|   +-- blogs/
|   |   +-- page.tsx          Blog listing
|   |   +-- write/page.tsx      Blog editor
|   |   +-- [slug]/page.tsx     Blog reader
```

```
|   |
|   +-- forums/
|   |   +-- page.tsx              Thread listing
|   |   +-- new/page.tsx          Create thread
|   |   +-- [id]/page.tsx         Thread + comments
|   |
|   +-- profile/
|   |   +-- [id]/page.tsx         Public profile view
|   |   +-- edit/page.tsx         Edit own profile
|   |
|   +-- search/page.tsx
|   +-- announcements/page.tsx
|   +-- dsa/page.tsx
|   +-- admin/page.tsx
|   +-- 403/page.tsx
|
+-- api/                          API route handlers
|   +-- auth/
|   +-- resources/
|   +-- blogs/
|   +-- forums/
|   +-- users/
|   +-- search/
|   +-- notifications/
|   +-- admin/
|
+-- components/                   All React components
|   |
|   +-- layout/
|   |   +-- Navbar.tsx
|   |   +-- Sidebar.tsx
|   |   +-- Footer.tsx
|   |
|   +-- auth/
|   |   +-- SignupForm.tsx
|   |   +-- LoginForm.tsx
|   |   +-- ForgotForm.tsx
|   |   +-- ResetForm.tsx
|   |
|   +-- resources/
|   |   +-- ResourceCard.tsx
|   |   +-- ResourceGrid.tsx
|   |   +-- ResourceFilter.tsx
|   |   +-- ResourceUploadForm.tsx
```

```
|   |
|   +-- blogs/
|   |   +-- BlogCard.tsx
|   |   +-- BlogEditor.tsx        (TipTap wrapper)
|   |   +-- BlogViewer.tsx        (sanitized HTML renderer)
|   |
|   +-- forums/
|   |   +-- ThreadCard.tsx
|   |   +-- CommentTree.tsx       (recursive nested comments)
|   |   +-- CommentItem.tsx
|   |   +-- CommentForm.tsx
|   |
|   +-- profile/
|   |   +-- ProfileHeader.tsx
|   |   +-- AchievementBadge.tsx
|   |   +-- ContributionStats.tsx
|   |   +-- ReputationBadge.tsx
|   |
|   +-- shared/
|       +-- VoteButton.tsx
|       +-- SearchBar.tsx
|       +-- FilterPanel.tsx
|       +-- MarkdownRenderer.tsx
|       +-- Avatar.tsx
|       +-- Tag.tsx
|       +-- Pill.tsx
|       +-- Modal.tsx
|       +-- Toast.tsx
|       +-- Skeleton.tsx
|       +-- NotificationBell.tsx
|       +-- NotificationPanel.tsx
|       +-- ReportButton.tsx
|       +-- AnnouncementBanner.tsx
|
+-- lib/                      Utility and service modules
|   +-- supabase/
|   |   +-- server.ts           Server-side Supabase client
|   |   +-- client.ts           Browser-side Supabase client
|   +-- validations/            Zod schemas for all inputs
|   +-- reputation.ts           Reputation calculation logic
|   +-- achievements.ts         Achievement unlock logic
|   +-- search.ts               Search query builder
|   +-- slugify.ts              Blog slug generator
|
```

```
+-- middleware.ts              Route protection
+-- tailwind.config.ts
+-- next.config.ts
```

## 7.2 State Management Strategy

There are two kinds of state in the application:

**Auth state** is global and managed via React Context. The Supabase session is read once on app load and stored in a context that all components can access. This tells every component whether the user is logged in and what their role is.

**Server data** (resources, blogs, threads, etc.) is managed by TanStack Query (React Query). It handles caching, background refetching, pagination, and optimistic updates. Every API call goes through a custom hook that wraps a React Query query or mutation.

Example hooks:

```
useResources(filters)        fetches paginated resource list
useResource(id)              fetches single resource
useCreateResource()          mutation: POST /api/resources
useVote(targetId, type)      mutation: POST /api/*/vote

useBlogs(filters)
useBlog(slug)
useCreateBlog()

useThreads(filters)
useThread(id)
useCreateThread()
useCreateComment()

useNotifications()           fetches + subscribes via Supabase Realtime
useCurrentUser()             wraps auth context
```

---

# 8. Feature Implementation Details

## 8.1 Full-Text Search

PostgreSQL full-text search is used across all content types. A `search_vector` column (type `TSVECTOR`) is maintained on the `resources`, `blog_posts`, and `forum_threads` tables. A database trigger updates this column automatically whenever a row is inserted or updated.

```
-- Example trigger for blog_posts
CREATE OR REPLACE FUNCTION update_blog_search_vector()
RETURNS TRIGGER AS $$
BEGIN
  NEW.search_vector :=
    setweight(to_tsvector('english', COALESCE(NEW.title, '')), 'A') ||
    setweight(to_tsvector('english', COALESCE(NEW.excerpt, '')), 'B') ||
    setweight(to_tsvector('english', COALESCE(array_to_string(NEW.tags, ' '), '')), 'C');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER blog_search_vector_update
  BEFORE INSERT OR UPDATE ON blog_posts
  FOR EACH ROW EXECUTE PROCEDURE update_blog_search_vector();

-- GIN index for fast full-text queries
CREATE INDEX blog_search_idx ON blog_posts USING GIN(search_vector);
```

Searching across all content types at once:

```
SELECT 'blog' AS type, id, title, slug AS ref
FROM blog_posts
WHERE search_vector @@ plainto_tsquery('english', $1)
  AND status = 'published'

UNION ALL

SELECT 'resource' AS type, id, title, id::text AS ref
FROM resources
WHERE search_vector @@ plainto_tsquery('english', $1)

UNION ALL

SELECT 'forum' AS type, id, title, id::text AS ref
FROM forum_threads
WHERE search_vector @@ plainto_tsquery('english', $1)

ORDER BY type;
```

## 8.2 Nested Forum Comments (Recursive Query)

Comments support two levels of nesting: a top-level comment and one level of replies. The full comment tree for a thread is fetched in a single recursive query:

```
WITH RECURSIVE comment_tree AS (
  -- Base case: top-level comments (no parent)
  SELECT
    c.*,
    0 AS depth
  FROM forum_comments c
  WHERE c.thread_id = $1
    AND c.parent_id IS NULL

  UNION ALL

  -- Recursive case: children of matched comments
  SELECT
    c.*,
    ct.depth + 1 AS depth
  FROM forum_comments c
  JOIN comment_tree ct ON c.parent_id = ct.id
  WHERE ct.depth < 1    -- limit to 1 level of nesting
)
SELECT * FROM comment_tree
ORDER BY depth ASC, created_at ASC;
```

The result is a flat list with depth information. The `CommentTree` component on the frontend arranges this into a nested visual structure.

## 8.3 Reputation System

Reputation points are awarded when specific actions occur. The point adjustment is applied to the `users.reputation` column after the triggering action is confirmed.

| **Action** | **Points** |
| --- | --- |
| Publish a blog post | +15 |

| Action | Points |
|---|---|
| Upload a resource | +10 |
| Create a forum thread | +5 |
| Your comment gets upvoted | +2 |
| Your resource gets upvoted | +3 |
| Your blog gets upvoted | +4 |
| Your blog gets downvoted | -1 |
| Your resource gets downvoted | -1 |
| Report confirmed by admin | -10 |

Reputation levels are derived from the current point total:

0 to 50 → Beginner
51 to 200 → Contributor
201 to 500 → Mentor
501+ → Expert

## 8.4 Achievement System

Achievements are checked asynchronously after user actions complete. They do not block or slow down the main request.

| Badge Key | Trigger Condition |
|---|---|
| first_blog | User publishes their first blog post |
| first_resource | User uploads their first resource |
| first_thread | User creates their first forum thread |
| ten_helpful | User receives 10 upvotes on comments |
| streak_7 | User has activity on 7 consecutive days |

| Badge Key | Trigger Condition |
|---|---|
| streak_30 | User has activity on 30 consecutive days |
| dsa_10 | User marks 10 DSA problems as solved |
| dsa_50 | User marks 50 DSA problems as solved |
| top_contributor | User reaches Contributor reputation level |
| mentor | User reaches Mentor reputation level |
| expert | User reaches Expert reputation level |

When a badge condition is met, a row is inserted into `achievements` with a `UNIQUE` constraint on `(user_id, badge_key)` ensuring each badge is only awarded once.

## 8.5 Notification Triggers

Notifications are inserted into the `notifications` table and delivered via Supabase Realtime, which pushes updates to the user's notification bell without requiring polling.

```
Event                       Recipient          Message
--------------------------  -----------------  ------------------------------
Reply to your thread        Thread author      "[Name] replied to your thread"
Reply to your comment       Comment author     "[Name] replied to your comment"
@mention in any content     Mentioned user     "[Name] mentioned you in [title]"
New announcement posted     All users          "[Title] - new announcement"
New resource in category    Subscribers        "New resource: [title]"
```

## 8.6 Blog Editor

The blog editor uses TipTap, configured with the following extensions:

```
StarterKit              (bold, italic, headings, lists, blockquote, HR)
CodeBlockLowlight       (code blocks with Shiki syntax highlighting)
Link                    (clickable hyperlinks)
Placeholder             (greyed out placeholder text)
CharacterCount          (shows word/character count)
```

Content is stored as sanitized HTML in the database. DOMPurify is used on the server before saving and on the client before rendering to prevent XSS attacks.

Slugs are generated from the blog title at creation time:

"My Placement Experience at Google" → "my-placement-experience-at-google"

If slug already exists:
"my-placement-experience-at-google-2"

---

# 9. Security Design

## 9.1 Row-Level Security (RLS)

RLS is enabled on every table. Supabase evaluates these policies at the database level, so even if an API route has a bug, the database itself will reject unauthorized reads or writes.

Table: blog_posts

```
Policy: "Anyone can read published posts"
  FOR SELECT USING (status = 'published')

Policy: "Authors can read their own drafts"
  FOR SELECT USING (author_id = auth.uid())

Policy: "Authors can insert their own posts"
  FOR INSERT WITH CHECK (author_id = auth.uid())

Policy: "Authors can update their own posts"
  FOR UPDATE USING (author_id = auth.uid())

Policy: "Admins can do anything"
  FOR ALL USING (
    EXISTS (
      SELECT 1 FROM users
      WHERE id = auth.uid() AND role = 'admin'
    )
  )
```

Similar policies are defined for every table with appropriate ownership and role checks.

## 9.2 Input Validation

Every API route validates its input using a Zod schema before processing anything. Validation runs on the server regardless of client-side validation.

Example schema for resource creation:

```
const createResourceSchema = z.object({
  title:        z.string().min(3).max(200),
  description:  z.string().max(2000).optional(),
  category:     z.enum(['notes','pdf','placement', ...]),
  semester:     z.number().int().min(1).max(8).optional(),
  subject:      z.string().max(100).optional(),
  tags:         z.array(z.string().max(30)).max(10).default([]),
  external_url: z.string().url(),
})
```

## 9.3 Content Sanitization

All user-generated HTML (from the blog editor and forum posts) is sanitized using DOMPurify before being stored in the database and again before being rendered in the browser. This prevents stored XSS attacks.

## 9.4 Rate Limiting

Basic rate limiting is applied at the Vercel middleware level using an in-memory counter (or Upstash Redis if free tier is needed). Write operations are limited per IP:

```
POST /api/resources   → 10 requests per minute per IP
POST /api/blogs       → 5 requests per minute per IP
POST /api/forums      → 10 requests per minute per IP
POST /api/comments    → 20 requests per minute per IP
```

## 9.5 Moderation Controls

| **Action** | **Who Can Do It** |
|---|---|
| Delete any content | Admin, Moderator |

| Action | Who Can Do It |
|---|---|
| Lock a thread | Admin, Moderator |
| Pin a thread | Admin, Moderator |
| Ban a user | Admin only |
| Dismiss a report | Admin, Moderator |
| Post an announcement | Admin, Moderator |
| Edit any user profile | Admin only |

---

# 10. Implementation Plan

## Phase 1 — Foundation (Weeks 1–4)

### Week 1: Project Setup and Deployment Pipeline

Tasks:
- Initialize Next.js project with TypeScript and App Router
- Configure Tailwind CSS and base design tokens
- Set up Supabase project (database, auth, RLS)
- Create all database tables and triggers from the schema above
- Connect Next.js to Supabase using @supabase/ssr
- Configure Vercel deployment with GitHub CI/CD
- Set up ESLint, Prettier, and Husky pre-commit hooks
- Create the base layout: Navbar, Sidebar, Footer skeleton

Deliverable: Deployed skeleton at a Vercel URL, database connected.

### Week 2: Authentication System

Tasks:
- Build signup page and form (name, email, password, batch)
- Build login page and form
- Build verify, forgot, and reset password pages
- Implement route protection middleware
- Add session-aware navbar (show name + logout when logged in)

- Implement the database trigger for auto-creating public.users
- Write Zod validation schemas for all auth inputs
- Test full auth flow end-to-end

Deliverable: Users can register, verify email, log in, log out,
and reset their password.

### Week 3: Resources Section

Tasks:
- Build resource listing page with category and sort filters
- Build resource detail page
- Build resource upload form (metadata + external URL)
- Implement tag filtering with GIN-indexed queries
- Implement upvoting (votes table + UI)
- Add basic PostgreSQL full-text search on resources only
- Implement reputation points on resource upload and upvote

Deliverable: Full resource upload and discovery experience.

### Week 4: Blog Section

Tasks:
- Integrate TipTap editor with code block and Shiki highlighting
- Build blog editor page (write + draft saving)
- Build blog listing page with tag filters
- Build blog reader page (sanitized HTML render)
- Implement draft vs published status
- Auto-generate slugs from title
- Add upvoting on blog posts
- Add full-text search on blog posts

Deliverable: Students can write, save drafts, publish, and read blogs.

---

## Phase 2 — Community Features (Weeks 5–8)

### Week 5: Forum System

Tasks:
- Build thread listing page with category filter and sort (hot/new/top)
- Build thread detail page with full nested comment tree

- Build create thread page
- Implement recursive comment fetching (CTE query)
- Build comment reply UI (2 levels of nesting)
- Implement upvoting on threads and comments
- Add lock and pin controls for moderators
- Add full-text search on forum threads

Deliverable: Full working forum with nested discussions.

## Week 6: Search and Notifications

Tasks:
- Build global search page combining resources, blogs, and forums
- Implement unified search query with UNION ALL
- Build notification bell component and panel
- Set up Supabase Realtime subscription for unread notification count
- Implement notification insertion on reply and mention events
- Parse @mentions in forum content and trigger notifications

Deliverable: Global search works. Users receive live notifications.

## Week 7: User Profiles and Reputation

Tasks:
- Build public profile page (bio, stats, achievements, activity)
- Build edit profile page (name, bio, avatar URL, batch)
- Implement reputation point tracking across all actions
- Display reputation level and badge on profile
- Implement achievement unlock logic and badge UI
- Build contribution stats section (blogs, resources, threads count)

Deliverable: Full user profile with reputation and achievements visible.

## Week 8: Moderation and Admin

Tasks:
- Build admin dashboard page (route-protected to admin role)
- Build report queue view with pending reports
- Implement report button on all content types
- Implement resolve and dismiss actions on reports
- Implement ban user (sets role to 'banned')
- Build announcement creation form
- Build announcements listing page

- Add announcement banner on home page for active announcements

Deliverable: Admins and moderators can moderate content and post announcements.

---

# Phase 3 — DSA and Growth (Weeks 9–12)

## Week 9: DSA Section

Tasks:
- Build DSA problem listing page (filter by topic, difficulty, platform)
- Implement mark as attempted / solved (dsa_progress table)
- Display user's progress on the listing page
- Highlight weekly challenge problems
- Build admin form to add/edit DSA problems

Deliverable: Structured DSA practice section with progress tracking.

## Week 10: Leaderboards and Gamification UI

Tasks:
- Build reputation leaderboard page (top contributors this week/all-time)
- Build DSA leaderboard (most problems solved)
- Implement activity streak tracking (daily action = streak day)
- Display streak on profile
- Polish achievement badge gallery on profile page

Deliverable: Leaderboards live. Streaks and gamification fully visible.

## Week 11: Discovery and Trending

Tasks:
- Build trending section on home dashboard (most upvoted this week)
- Add "related posts" on blog and forum pages (shared tags)
- Add personalized feed based on user's most-used tags
- Improve filter UX across all listing pages

Deliverable: Home dashboard shows meaningful personalized content.

## Week 12: Testing and Hardening

Tasks:
- Write end-to-end tests for core flows (signup, upload, post, comment) using Playwright
- Audit all DB queries for performance (add missing indexes)
- Run Lighthouse audit on key pages, fix performance issues
- Verify all RLS policies are correctly blocking unauthorized access
- Review and tighten rate limiting rules
- Write contributor documentation (README, setup guide, DB schema docs)

Deliverable: Stable, tested, documented platform ready for community use.

---

## Phase 4 — Future Additions (Post Week 12)

These features require either additional infrastructure or significant effort and are planned for after the core platform is stable.

| **Feature** | **Notes** |
|---|---|
| Integrated Online Judge | Use Judge0 (self-hostable on Oracle Cloud VPS) |
| AI Search | Supabase pgvector for semantic search |
| Recommendation Engine | Collaborative filtering on resource interactions |
| Placement Dashboard | Company-wise grouping, offer tracker |
| Email Digest | Weekly summary of top content |

---

# 11. Free Tier Limits and Constraints

## 11.1 Supabase Free Tier

| **Limit** | **Amount** | **Impact** |
|---|---|---|
| Database storage | 500 MB | Sufficient for text; files on external links. |

| Limit | Amount | Impact |
|---|---|---|
| Auth users | 50,000 | Far beyond DecSync batch size. |
| Realtime connections | 200 | More than enough concurrently. |
| Outbound emails | 3 per hour | Fine for small community signups. |
| API requests | Unlimited | No concern. |

## 11.2 Vercel Free (Hobby) Tier

| Limit | Amount | Impact |
|---|---|---|
| Bandwidth | 100 GB/month | Sufficient. No video/file hosting. |
| Serverless runtime | 100 GB-hours | Sufficient for API routes. |
| Build minutes | 6,000/month | More than enough. |
| Domains | Unlimited | Can use custom .tech domain free. |

## 11.3 When to Upgrade

The platform will not need paid services until one of the following happens:

- The platform expands beyond the DecSync batch to the entire college or multiple batches (500+ active users)
- Direct file hosting becomes necessary instead of external Drive links
- Email digest features require more than 3 emails/hour consistently

At that point, Supabase Pro ($25/month) and Vercel Pro ($20/month) are the only two upgrades needed. Everything else in the stack remains free regardless of scale.